Help

```
:help expression-commands  Build-in commands, including let, unlet
```

Basics

```
"" Command call
echo "Hello"

" call with variable/function
echo g:x                        " OK
echo f()                        " OK
buffer g:x                      " NO. Some commands wrap the arguments in quotes
                                " i.e. buffer "g:x"
buffer f()                      " NO
exec "buffer " . g:x            " OK
exec "buffer " . f()            " OK


" Variables
g:x = echo "Hello"              " ERROR. Cannot assign command result to
variable
```

Commands

```
" echo - Print to screen
"
echo g:myvar


"" let - Set and get variables
"
let                   Print all variables
let t:x = 1           Set variable
let t:x               Get t:x
let y                 ERROR. undefined variable


" call - call a function
"
call myfunction()


"" execute {expr}
" execute a command from an expression
"
execute "ls %"          Same as ls %
exec "ls %"             Same
execute ls              ERROR. Not an expression

"Multiple arguments are concatenated with spaces
execute "ls" "%"          Same as echo 'Hello'.

" Pipes are not an argument
execute "ls" "%" | echo Hello


"" set - options
"
set                   Options with non-default
set all               Print all options
```

```
"" function -
function              List defined functions
function /^myfn/      List functions matching pattern
function myfn         Print myfn (function body)


"" scriptnames - print loaded files
"
" Includes: vimrc, syntax files
scriptnames           Loaded files, including vimrc, syntax files, etc.


"" ! - run shell command
"
! ls                  Shell command


"" cd - change directory
""
cd %:p:h


"" Print last error message
message                Last error messages


"" source - read commands from file
so[urce] {file}        Read commands from file
```

File commands

```
"" print - print lines
"
print                Print current line
p                    Same
p 10                 Print 10 lines from current line
1,10p                Print range. Line 1~10.


"" copy - copy lines to
"
copy 10          Copy current line and paste below line 10 (i.e. line 11)
copy 0           Copy to top of file
copy $           Copy to bottom of file
1,5copy 0        Copy range to top of file


"" move - move lines to
" Similar to copy, but moves lines instead of copying
move 10          Move current line to below line 10 (i.e. line 11)
move 0           Move to top
move $
1,5move 0


"" normal - apply normal mode command
"
normal iHello        Same as insert mode -> Hello on current line
normal @a            Apply @a macro on current command

1,5normal @a         Apply on range.
'<,'>normal @a       Apply on visual selection.
```